SQL Lab 3 – SQL against Dimensional Data Model

(requires access to ODAA Metrics models in the ODAAMETRICS schema)

In order to complete this lab, you will need to connect to the class database (INFO4555) via the SQL Developer application found on your AppStream. Details on how to connect to the class database may be found in document: *"Connecting to class database via SQL Developer",* in the project files folder of the class Canvas site.

The first star we will query is called "Data Mart Records". This is a simple dimensional data model which tracks the growth of the warehouse environment. The fact table "Datamart_Records_fc" is a periodic snapshot fact which contains the record count of each table, in each data mart, on the first day of each month. The only fact in the datamart_records_fc table is row_count. However, you will see there is a lot we can do with even a simple dimensional model.

Model Name: Data Mart Records

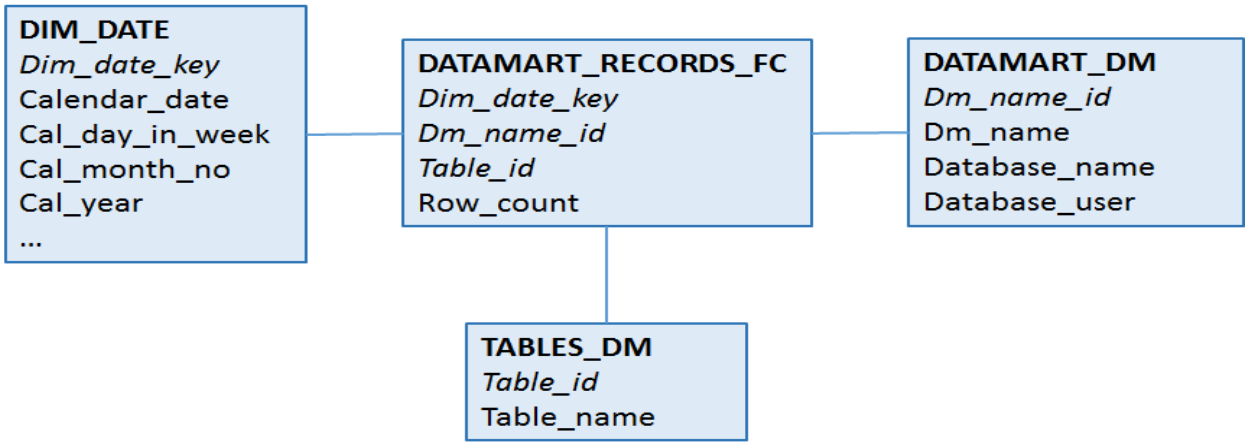
Grain: One record per production data mart, per data mart table, per month

Filters: by data mart, table, year, month

Measures:

Number of records by table, identified as row_count

Data Mart Records - Data Model



SQL Lab 3 – SQL against Dimensional Data Model

(requires access to ODAA Metrics models in the ODAAMETRICS schema)

We will start with a simple query to look at the datamart_dm table.

Copy the SQL and paste it into your SQL Developer worksheet. Once your SQL is in the worksheet, click the big green arrow to execute the SQL.

```
select *
from ODAAMETRICS.datamart_dm;
```

This is a very simple query, there are no filters or table joins. The "select *" is using a * wildcard to indicate list all columns. The "ODAAMETRICS." Before the table name, is specifying the schema the table lives in. For this exercise, all of the tables live in the "ODAAMETRICS" schema, so all tables listed should have this prefix, or you will receive a "table or view does not exist error".

Now let's add a simple filter to our query.

```
select *
from ODAAMETRICS.datamart_dm
where dm_name like '%PROD%';
```

In this example, I used the LIKE operator with the % wild cards to filter on dm_names with the word PROD in the name. This gives me a list of dm_names that are production systems.

Let's add one more filter. We will add a filter on database_user.

```
select *
from ODAAMETRICS.datamart_dm
where dm_name like '%PROD%'
and database_user = 'WDDW';
```

Q1) Now we are down to a single result set. What is the DM_NAME for the production system which uses the database_user WDDW?

Let's take a quick look at our fact table.

```
select *
from ODAAMETRICS.datamart_records_fc;
```

Hopefully the result set looks like some of the examples we have seen in class for fact tables. Basically a list of foreign keys and a measure. All numeric values. Not very useful until you join the fact table to the dimension tables.

Now let's try a query against the datamart_records model. We will join to all of its related dimensions and filter by calendar year = 2015 and calendar month number = 3. Notice all of the joins between the Dimension table primary keys and fact table foreign keys are simple (inner) joins. A good dimensional data model should only require inner joins.

```
select dt.cal_year, dt.cal_month_no, t.table_name, rf.row_count
from ODAAMETRICS.datamart_records_fc rf, ODAAMETRICS.dim_date dt,
ODAAMETRICS.datamart_dm dm, ODAAMETRICS.tables_dm t
where rf.dim_date_key = dt.dim_date_key
and rf.dm_name_id = dm.dm_name_id
and rf.table_id = t.table_id
and dt.cal_year = 2015
and dt.cal_month_no = 3
;
```

A few things to note about this query.

1) We provided a list of columns, versus using a * wildcard. When you join all of the tables there are many more columns than we care to see in a single result set, so we list the columns we are interested in seeing on the SELECT line.
2) Notice the use of an alias after each table in the FROM statement. You can name the alias whatever you like, as long as they are unique to the table they refer to. Notice the use of the alias in the SELECT line and WHERE clause. The use of a table alias greatly reduces the amount of typing and size of the SQL statement.
3) In the WHERE clause we start by defining all of the table joins.
4) Later in the WHERE clause we add any needed filters, in this case calendar year and calendar month number.

The above query returns 778 records, so there were 778 tables in the data warehouse environment on March 1, 2015. This query is returning the records at the most atomic level, which is not very useful, if we want to know how many total records are in the warehouse, or a particular "DM_NAME".

Now let's find out how many records were in the warehouse in March 2015. This time we will be aggregating the data by year and month.

```
select dt.cal_year, dt.cal_month_no, sum(rf.row_count)
from ODAAMETRICS.datamart_records_fc rf, ODAAMETRICS.dim_date dt,
ODAAMETRICS.datamart_dm dm, ODAAMETRICS.tables_dm t
where rf.dim_date_key = dt.dim_date_key
and rf.dm_name_id = dm.dm_name_id
and rf.table_id = t.table_id
and dt.cal_year = 2015
and dt.cal_month_no = 3
group by dt.cal_year, dt.cal_month_no
order by dt.cal_year, dt.cal_month_no
;
```

A few things to note about this query:

1) The SELECT line is reduced to the columns we are GROUPING BY a.k.a (aggregating by), plus the measure affected by the aggregation. In this case we are using a SUM function for the row_count column.
2) Notice after the WHERE clause, we have added a GROUP BY statement which lists the columns to use for the aggregation.
3) Finally we have an ORDER BY clause. In this case we have filtered down to a single year month combination, so there will be only one record returned, however when we open up the month filter, we will see the usefulness of the order by.

Q2) How many records were in the Data Warehouse on March 2015?

Now let's look at the record counts over a few months:

```
select dt.cal_year, dt.cal_month_no, sum(rf.row_count)
from ODAAMETRICS.datamart_records_fc rf, ODAAMETRICS.dim_date dt,
ODAAMETRICS.datamart_dm dm, ODAAMETRICS.tables_dm t
where rf.dim_date_key = dt.dim_date_key
and rf.dm_name_id = dm.dm_name_id
and rf.table_id = t.table_id
and dt.cal_year = 2015
--and dt.cal_month_no = 3
group by dt.cal_year, dt.cal_month_no
order by dt.cal_year, dt.cal_month_no
;
```

SQL Lab 3 – SQL against Dimensional Data Model

(requires access to ODAA Metrics models in the ODAAMETRICS schema)

A few things to note about this query:

1) The only change made was to comment out the month filter. "—" (two dashes) before a line in SQL is the comment notation. Now we see the aggregated results for all 12 months of 2015.

Q3) How many records were in the Data Warehouse in June 2015?

Now let's try aggregating by the year 2015, by removing the month column and month from the group by.

```
select dt.cal_year, sum(rf.row_count)
from ODAAMETRICS.datamart_records_fc rf, ODAAMETRICS.dim_date dt,
ODAAMETRICS.datamart_dm dm, ODAAMETRICS.tables_dm t
where rf.dim_date_key = dt.dim_date_key
and rf.dm_name_id = dm.dm_name_id
and rf.table_id = t.table_id
and dt.cal_year = 2015
--and dt.cal_month_no = 3
group by dt.cal_year
order by dt.cal_year
;
```

A few things to note about this query:

1) The month filter remains commented out, so records are returning for January – December
2) I simply removed the month from the SELECT line, Group BY, and Order by.
3) The query is now returning the sum of the row counts for each table, for each snapshot (month) of the year 2015.

Q4) What is the record count returned by this query?

Q5) Why does this number not accurately reflect the number of records in the data warehouse?

SQL Lab 3 – SQL against Dimensional Data Model

(requires access to ODAA Metrics models in the ODAAMETRICS schema)

Let's go back to our query which filters (limits) by year 2015 and month 3. This time, however, we will "drill down" a level in our aggregation to expose the record counts by DM_NAME. A.K.A Data Mart Name

```
select dt.cal_year, dt.cal_month_no, dm.dm_name, sum(rf.row_count) total_row_count
from ODAAMETRICS.datamart_records_fc rf, ODAAMETRICS.dim_date dt,
ODAAMETRICS.datamart_dm dm, ODAAMETRICS.tables_dm t
where rf.dim_date_key = dt.dim_date_key
and rf.dm_name_id = dm.dm_name_id
and rf.table_id = t.table_id
and dt.cal_year = 2015
and dt.cal_month_no = 3
group by dt.cal_year, dt.cal_month_no, dm.dm_name
order by dt.cal_year, dt.cal_month_no, dm.dm_name
;
```

A few things to note about this query:

1) Cal_month_no = 3 is no longer commented out, so we are filtering by the month of March
2) Dm_name has been added to the SELECT line
3) Dm_name has been added to the group by and order by
4) Total_row_count has been added as a column name alias for "sum(rf.row_count)". This is a more user friendly column header.

Q6) What is the DM_NAME with the lowest Total_row_count in March 2015?

Now let's "drill down" another level, to see the record counts by table in a single data mart. We will filter by DM_name = 'KFSDM PROD AWS' and add table_name to our aggregation "GROUP BY".

```
select dt.cal_year, dt.cal_month_no, dm.dm_name, t.table_name, sum(rf.row_count) total_row_count
from ODAAMETRICS.datamart_records_fc rf, ODAAMETRICS.dim_date dt,
ODAAMETRICS.datamart_dm dm, ODAAMETRICS.tables_dm t
where rf.dim_date_key = dt.dim_date_key
and rf.dm_name_id = dm.dm_name_id
and rf.table_id = t.table_id
and dt.cal_year = 2015
and dt.cal_month_no = 3
and dm.dm_name = 'KFSDM PROD AWS'
group by dt.cal_year, dt.cal_month_no, dm.dm_name, t.table_name
order by dt.cal_year, dt.cal_month_no, dm.dm_name, t.table_name
;
```

A few things to note about this query:

1) Table_name was added to the SELECT line

2) Table_name was added to the GROUP BY and ORDER BY clauses
3) A filter was added dm_name = 'KFSDM PROD AWS' to reduce the dataset to a single data mart.

Q7) How many records were in the GL_DETAIL_FC table in March 2015?

Now we want to better understand the growth of the GL_DETAIL_FC table. So, let's add a filter of table = 'GL_DETAIL_FC' and remove the year and month filter.

```
select dt.cal_year, dt.cal_month_no, dm.dm_name, t.table_name, sum(rf.row_count) total_row_count
from ODAAMETRICS.datamart_records_fc rf, ODAAMETRICS.dim_date dt,
ODAAMETRICS.datamart_dm dm, ODAAMETRICS.tables_dm t
where rf.dim_date_key = dt.dim_date_key
and rf.dm_name_id = dm.dm_name_id
and rf.table_id = t.table_id
--and dt.cal_year = 2015
--and dt.cal_month_no = 3
and dm.dm_name = 'KFSDM PROD AWS'
and t.table_name = 'GL_DETAIL_FC'
group by dt.cal_year, dt.cal_month_no, dm.dm_name, t.table_name
order by dt.cal_year, dt.cal_month_no, dm.dm_name, t.table_name
;
```

A few things to note about this query:

1) The cal_year and cal_month_no filters were commented out, to open up a longitudinal view of the data.
2) Added a filter for table_name = 'GL_DETAIL_FC' , this limits the result set to only this table, but over time.

Q8) What is the record count for GL_DETAIL_FC in January **2014**?

SQL Lab 3 – SQL against Dimensional Data Model

(requires access to ODAA Metrics models in the ODAAMETRICS schema)

The next data model tracks the time it takes the ETL processes to run.

Model Name: Load Times

Grain: One record per successful load per production data mart.

Filters: by data mart, Date(year, month, day)

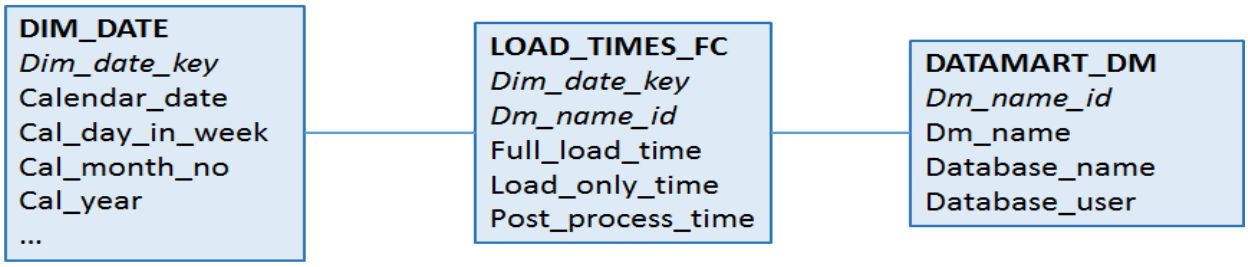
Measures:

Full_load_time – number of minutes to complete the entire load process

Load_only_time – number of minutes to complete the ETL processing only

Post_processing_time – number of minutes to complete statistics gathering and index builds

Load Times - Data Model



The final queries will be against the "Load Times" data model. Use the diagram above, and the SQL syntax examples, from the prior examples, to write the SQL needed to answer the final questions.

SQL Lab 3 – SQL against Dimensional Data Model

(requires access to ODAA Metrics models in the ODAAMETRICS schema)

Here is a query to get you started:

```
select dt.cal_year, dt.cal_month_no, dt.cal_month_name, dm.dm_name, SUM(lt.full_load_time),
count(1) num_of_loads,
  ROUND((SUM(lt.full_load_time)/count(1))) full_load_time_avg,
  MAX(lt.full_load_time) longest_full_load_time, MIN(lt.full_load_time) shortest_full_load_time
from ODAAMETRICS.load_times_fc lt, ODAAMETRICS.dim_date dt, ODAAMETRICS.datamart_dm dm
where lt.dim_date_key = dt.dim_date_key
and  dm.dm_name_id = lt.dm_name_id
and dt.cal_year = 2015
and dt.cal_month_no = 3
--and dm.dm_name = 'SA PROD'
group by dt.cal_year, dt.cal_month_no, dt.cal_month_name, dm.dm_name
order by dt.cal_year, dt.cal_month_no, dm.dm_name;
```

A few things to note about the above query.

1. The grain of the fact table is at the individual load occurrence. However, we are aggregating by year/month/ and data mart (dm_name).
2. We are filtering by year 2015 and month 3 (March)
3. We are using a few different functions in the SELECT clause. SUM, MAX, MIN, & ROUND
4. We are also deriving the average full_load_time by dividing the sum of the load times, by number of loads which occurred for that period.
5. Finally, several of the columns in the SELECT clause have been assigned aliases to provide more user-friendly result set headers.

Q9) What was the longest “full_load_time” during the month of March, 2015?

Q10) Which Data Mart (DM_NAME) had the lowest average full load time for February 2013?

Q11) What was the shortest_full_load_time for the CR PROD data mart February 2015?

SQL Lab 3 – SQL against Dimensional Data Model

(requires access to ODAA Metrics models in the ODAAMETRICS schema)

Q12) Write the SQL to produce the following result set. *(The column names, values, and record order should match). Note the full dataset returned contains only the 12 records shown below.*

	CAL_YEAR	CAL_MONTH_NO	CAL_MONTH_NAME	DM_NAME	TOTAL_POST_PROCESS_TIME	NUM_OF_LOADS	POST_PROCESS_TIME_AVG
1	2014	1	JAN	SA PROD	1777	21	85
2	2014	2	FEB	SA PROD	1628	19	86
3	2014	3	MAR	SA PROD	1597	20	80
4	2014	4	APR	SA PROD	1716	21	82
5	2014	5	MAY	SA PROD	1655	20	83
6	2014	6	JUN	SA PROD	1832	22	83
7	2014	7	JUL	SA PROD	1985	23	86
8	2014	8	AUG	SA PROD	1780	20	89
9	2014	9	SEP	SA PROD	1761	19	93
10	2014	10	OCT	SA PROD	2247	23	98
11	2014	11	NOV	SA PROD	2018	20	101
12	2014	12	DEC	SA PROD	2407	23	105

Copy/paste the SQL you used to come up with the above answer, onto your answer worksheet.

SQL12) *Past SQL in your worksheet.*